

Task Identifiers and Task Execution

Contents

- [Data Tasks](#)
- [Local Tasks](#)
- [Control Tasks](#)

The term “task identifier” or “task ID” is used to refer to a numerical value that can be associated with a task. A task ID is associated with a task through a process called “binding” that is carried out by special builtins depending on the type of task.

There exist three types of tasks, each with an associated task ID handle type.

These three types of tasks are:

- Data Tasks
- Local Tasks
- Control Tasks

The following three sections explain the semantics of each task type, how they can be bound to task IDs, and how they can be scheduled for execution.

Data Tasks

Data tasks are wavelet-triggered tasks (WTTs) that are associated with a `data_task_id`.

On WSE-2, a `data_task_id` is constructed from a routable identifier known as a `color` (see [Routable Identifiers \(Colors\)](#)) using `@get_data_task_id` (see [@get_data_task_id](#)), and can be bound to a data task using `@bind_data_task` (see [@bind_data_task](#)).

On WSE-3, a `data_task_id` is constructed from a hardware queue known as an `input_queue`, also using `@get_data_task_id`.

We demonstrate constructing a data task ID and binding it to a task in the code block below:

```

const iq = @get_input_queue(2); // Input queue must be in range 0 to 7
const rc = @get_color(12); // Routable color must be in range 0 to 23

// On WSE-2, construct task ID from routable color ID
// On WSE-3, construct task ID from input queue ID
const task_id =
  if (@is_arch("wse3")) @get_data_task_id(iq)
  else                  @get_data_task_id(rc);

// Bind data task to ID
// On WSE-2, task ID will be 12; on WSE-3, task ID will be 2
@bind_data_task(my_task, task_id);

// On WSE-3, associate input queue with a color
if (@is_arch("wse3")) @initialize_queue(iq, .{ .color = rc });

```

Routable Identifiers (Colors)

On WSE-2 and WSE-3, the IDs 0 to 23 (inclusive) are recognized by the hardware as virtual communication channels for passing wavelets between PEs. These IDs are sometimes referred to as the routable “colors” and can be constructed using [@get_color](#) (see [@get_color](#)).

Note that these IDs are the only ones that can be used with the [@set_color_config](#) and [@set_local_color_config](#) builtins (see [@set_color_config](#), [@set_local_color_config](#)).

Execution Semantics

A data task can be scheduled for execution if and only if it is bound to a data task ID that is activated and unblocked.

A data task is activated by receiving a wavelet along a given color. On WSE-2, that color (or routable identifier) is the color that was used to create the [data_task_id](#) bound to the respective data task. On WSE-3, that color is the color to which the associated [input_queue](#) is bound.

A data task must have at least one input argument representing the wavelet’s payload. If it has more than one input argument then the wavelet’s payload is equally split among those input arguments.

All data task IDs that are bound to a data task are initially unblocked. They can be explicitly blocked using [@block](#) (see [@block](#)) during the evaluation of a top-level comptime block or at runtime.

Once blocked, a data task ID can be unblocked explicitly using [@unblock](#) (see [@unblock](#)) or upon completion of asynchronous fabric DSD operations (see [Completion of Asynchronous DSD Operations](#)).

Local Tasks

Local tasks, or self-activated tasks, are associated with a [local_task_id](#), which is constructed from an activatable identifier (see [Activatable Identifiers](#)) using [@get_local_task_id](#) (see [@get_local_task_id](#)) and then bound to a local task using [@bind_local_task](#) (see [@bind_local_task](#)).

Activatable Identifiers

The IDs 0 to 30 (inclusive) on WSE-2, or 8 to 30 (inclusive) on WSE-3, can be used to construct `local_task_id` values that can then be used to activate local tasks, which is why they are referred to as “activatable” identifiers.

IDs 29 and 30 should generally be avoided in programs as they are used for system tasks. ID 29 is bound to a teardown task that runs when the tile is in teardown mode and ID 30 is bound to a timer task.

Execution Semantics

A local task can be scheduled for execution if and only if it is bound to a local task ID that is activated and unblocked.

Local tasks accept no input arguments and can be activated explicitly using the `@activate` builtin (see [@activate](#)) or upon completion of an asynchronous fabric (see [Completion of Asynchronous DSD Operations](#)) or FIFO (see [Task Activation on Pop and Push](#)) DSD operation.

All local task IDs that are bound to a local task are initially unblocked. They can be explicitly blocked using `@block` (see [@block](#)) during the evaluation of a top-level comptime block or at runtime.

Once blocked, a local task ID can be unblocked explicitly using `@unblock` (see [@unblock](#)) or upon completion of asynchronous fabric DSD operations (see [Completion of Asynchronous DSD Operations](#)).

Control Tasks

Control tasks are control wavelet-triggered tasks that are associated with a `control_task_id`, which is constructed from a control identifier (see [Control Identifiers](#)) using `@get_control_task_id` (see [@get_control_task_id](#)) and then bound to a control task using `@bind_control_task` (see [@bind_control_task](#)).

Control Identifiers

On WSE-2 and WSE-3, the IDs 0 to 63 (inclusive) can be used to create `control_task_id` values.

Execution Semantics

A control task bound to a `control_task_id` value `CID` is scheduled for execution if and only if the following two conditions are met:

- A control wavelet carrying the `CID` value in its payload is received.
- The communication channel that is used to receive the aforementioned control wavelet is unblocked.

The second condition needs to be explicitly satisfied using `@unblock` (see [@unblock](#)) during the evaluation of a top-level comptime block or at runtime. The input to `@unblock` should be the routable identifier (value of type `color`) associated with the control wavelet’s input communication channel.

Note that if the same communication channel is bound to a data task as well, then this unblocking is not necessary.

The payload of a control wavelet consists of the `control_task_id` value and a data section. Both can be passed as input arguments to the control task.

